

SpaceSails: Secretly a Classroom

Deterministic Real-Time Orbital Simulation in the Browser — and the Human-PO / AI-Head-Coder Method That Built It

Erno Soinila, D.Sc. (Eng.)

Senior AI Engineer, Efima — Product Owner

`erno.soinila@efima.com`

Claude (Fable 5)*

Head Coder

Anthropic

DRAFT v0.3 — July 2026

Abstract. SpaceSails is a browser-native solar-system sailing and piracy game whose engine is a deterministic 2D N -body integrator compiled to WebAssembly. Nothing in it cheats: every trajectory is integrated, every prediction is honestly uncertain, and the hardest maneuvers are hard because physics says so. The same properties that make the game honest make it a classroom: the repository ships a numerical-methods lab course — now roughly thirty type-it-in lessons — that runs on the game’s own engine, and the game ships *the numbers the labs measure*. This is the method we most want to convey: measure a mechanic in a probe first, then ship the measured number — the autopilot’s orbit-keeping trim budget, the wolves’ pursuit-escape envelope, the harbor-approach corridor, the mass-driver timetable — so that *every number in the game is a number a probe printed*. We describe (1) the system: a determinism-first core, adaptive symplectic integration with exact maneuver-node landing, honest uncertainty cones, and a closed-form no-tunneling proximity test; (2) the game that grew on it — a ship that narrates its own flight, autopilot that *keeps* an orbit at a quoted trim cost, deliberate piracy resolved by on-screen dice, buried treasure with generated maps, a favor bank, and a tolerant save vault; (3) the duty-station interface that distributes one simulation across eight crew desks; and (4) the process: a two-person team — a human product owner and an AI “head coder” who *orchestrates* multiple implementation agents working in parallel across isolated git worktrees — that took the project from empty repository to 165 reviewed, tested, merged pull requests across 12 active coding days (with travel breaks in between), the owner approving from a phone and the AI planning, reviewing, verifying in the owner’s own live browser, and running the merge train. We argue the combination — deterministic simulation as both gameplay substrate and curriculum, plus orchestrated AI implementation under tight human product direction — is a repeatable method, and we report what it cost, where it failed, and what the failures taught the game.

1 Introduction

On the surface, SpaceSails is a piracy game: solar sails, cargo pods, hunters, a gun deck. Underneath, it is a numerical-methods and orbital-mechanics classroom, and the disguise is deliberate. “Secretly edutainment” is the project’s design law, not an accident of scope: the owner’s first shipped software (SoittoPeli, 2000) was music edutainment for children, and SpaceSails carries the same DNA on purpose.

Our thesis is that *honesty* is the unifying design constraint, and that it pays for itself three times over. The simulation never lies: it is bit-deterministic, so replays, tests, and lessons are the same artifact (§2). The instruments never lie: prediction cones admit exactly what they do not know, and that admission is itself a game mechanic (§6.2). And the curriculum never lies: every number printed in the lab course comes from running that lesson’s probe against the shipping engine, which makes the course an adversarial test suite the engine must survive (§6). Play the game well and you have quietly learned the real thing.

This paper contributes:

1. a determinism-first architecture for real-time orbital simulation in WebAssembly, with the concrete rules that kept client and (archived) multiplayer server bit-identical;
2. honest uncertainty as a game mechanic: prediction cones whose growth law was *falsified by the game’s own lab course* and then fixed (§6.2);
3. fire control as pedagogy: the shooting method for boundary value problems — damped Newton iteration over launch bearing and ejection charge, with every residual evaluated by one flight of the real integrator — surfaced as a visible, converging CALCULATING FIRING SOLUTION moment (§6.3);
4. the *lab-to-game pipeline* as a repeatable R&D method (§6.4): a growing ladder of console probes that measure a mechanic honestly, after which the game ships the measured number rather than a tuned guess — orbit-keeping trims, the pursuit-escape cliff, harbor corridors, and a mass-driver timetable, each traced to the probe that printed it;

*Listed to record the head-coder role faithfully. Publication venues (e.g. ACM) do not permit AI systems as authors — authorship requires the capacity to take responsibility for the work — so on submission this credit becomes an extended acknowledgment with the same content.

5. an experience report of an *orchestrated* human-PO / AI-head-coder method (§7): one lead model that plans, reviews, and merges while several implementation agents work in parallel isolated worktrees, plus a live-browser triage loop in which the AI reads and repairs the owner’s own running tab mid-playtest, with the working agreements — and failure modes — that made review at content-per-day scale possible.

Because the entire project lives in one public repository built at high cadence, its git history is unusually complete: every model decision, every rejected alternative, and every owner correction is a commit message. Section 3 exploits this to give an autobiographical account of how the gravity model of §2 actually came to be.

2 The Deterministic Core

2.1 Determinism is law

No wall clock, no Random, no environment-dependent math anywhere in SpaceSails.Core. Every state transition is a pure function of the prior state and a fixed timestep schedule. The consequences compound: replays are free; the archived multiplayer server agreed with clients bit-for-bit; and — decisive for this paper — *tests and lessons are the same artifact*. A lab probe is just a test that prints its evidence.

2.2 Dynamics and integration

Celestial bodies ride rails: an ephemeris interface gives position as a pure function of time, $\mathbf{r}_k(t)$. Ships are test particles feeling point-mass gravity from every body,

$$\mathbf{a}_i(\mathbf{r}_i, t) = \sum_k \mu_k \frac{\mathbf{r}_k(t) - \mathbf{r}_i}{\|\mathbf{r}_k(t) - \mathbf{r}_i\|^3}, \quad \mu_k = Gm_k, \quad (1)$$

integrated with semi-implicit (symplectic) Euler:

$$\begin{aligned} \mathbf{v}_{n+1} &= \mathbf{v}_n + \mathbf{a}(\mathbf{r}_n, t_n) \Delta t_n, \\ \mathbf{r}_{n+1} &= \mathbf{r}_n + \mathbf{v}_{n+1} \Delta t_n. \end{aligned} \quad (2)$$

The velocity-first update is what buys long-horizon energy behavior at first-order cost: the scheme is symplectic, so orbits precess rather than spiral, and a multi-week projection remains a trustworthy *shape* even where it is not a trustworthy *ephemeris*.

2.3 Adaptive stepping with exact node landing

The step size follows the local dynamical time of the nearest dominant attractor,

$$\Delta t_n = \text{clamp}\left(\eta \min_k \sqrt{\frac{d_{ik}^3}{\mu_k}}, \Delta t_{\min}, \Delta t_{\max}\right), \quad (3)$$

where $d_{ik} = \|\mathbf{r}_k - \mathbf{r}_i\|$ and η is a safety fraction: small steps in deep gravity wells, long strides in interplanetary cruise.

One further rule makes plotted maneuvers exact rather than approximately timed: the step is truncated to land precisely on every maneuver node’s timestamp,

$$\Delta t_n \leftarrow \min(\Delta t_n, t_{\text{node}} - t_n), \quad (4)$$

so a plotted burn executes in the projection at the same instant the live loop fires it. Universal-variable Kepler propagation and patched conics were rejected on purpose: they assume one attractor per arc and disagree with the integrator exactly where the game happens — flybys. (Lab 06 §C quantifies what adaptive stepping still misses at periapsis and what it would cost to fix.)

2.4 The no-tunneling rule

Closest-approach queries and ordnance-hit tests never sample: within each integrator step the relative motion of two objects is piecewise linear, $\mathbf{p}(s) = \mathbf{p}_0 + s\mathbf{w}$ with $s \in [0, 1]$, where $\mathbf{p}_0$ is the relative position at the start of the step and $\mathbf{w} = \Delta\mathbf{v}\Delta t$ the relative displacement across it. The minimum distance has a closed form:

$$s^* = \text{clamp}\left(-\frac{\mathbf{p}_0 \cdot \mathbf{w}}{\|\mathbf{w}\|^2}, 0, 1\right), \quad d_{\min} = \|\mathbf{p}_0 + s^*\mathbf{w}\|. \quad (5)$$

Quadratic algebra, so *no step size can tunnel a fast graze through a target*. This replaced a per-sample check that the lab course itself proved dishonest (§6.2).

2.5 WebAssembly performance envelope

The client runs interpreted IL in the browser at roughly $100\times$ native cost, and the architecture is shaped by that number rather than in denial of it: NPC route search integrates at one-day steps, live NPCs advance in 60-second quanta, and the high-warp path adapts quantum length to time compression. Determinism makes these coarsenings safe to test: the coarse and fine paths are compared step-for-step in the suite.

3 Gravity at Speed in a Browser: How the Model Happened

Section 2 described the model as if it had been designed in one sitting. It was not. It accreted, one falsified assumption at a time, and the repository’s git history records every step. This section replays that history in first person — the head coder’s account of the thoughts that led to the model the Core runs today.

3.1 Day one: choose boring, test bitwise (M1)

The first physics commit made four decisions in an afternoon, all of them deliberately boring. Double precision and a 2D ecliptic plane, because the game is a map and doubles are what WebAssembly does natively. Planets on rails — position as a pure function of time — because planets do not care

about ships, and rails make the N -body problem a one-body problem N times. Ships as test particles feeling Eq. (1). And semi-implicit Euler, Eq. (2), at a fixed one-second step.

The integrator choice deserves the honesty it was made with. Explicit Euler was never a candidate: it injects energy and orbits visibly spiral outward within days. Runge–Kutta 4 was the respectable alternative — fourth-order accurate — but it costs four gravity evaluations per step and is not symplectic, so its orbits decay *slowly*, which is worse than decaying visibly: the error hides until a two-year plot exposes it. Semi-implicit Euler is first-order, crude, and symplectic — it conserves a nearby Hamiltonian, so orbits precess instead of spiraling. For a game whose long projections must be trustworthy *shapes*, phase error is survivable and energy error is not. The commit landed with the test that made the choice accountable: a 30-day circular orbit holding radius to $< 0.5\%$, and a bitwise-identical replay test that has guarded every physics change since.

3.2 Plotting mode breaks the fixed step (M4)

The fixed one-second step survived exactly three milestones. Plotting mode needed to draw a 60-day trajectory ribbon — later 730 days — and at $\Delta t = 1$ s that is 6×10^7 integration steps per replot, in a browser, in interpreted IL. The observation that rescued it is the oldest one in celestial mechanics: nothing happens in deep space. The local dynamical time $\sqrt{d^3/\mu}$ is the natural clock of the problem, so the step became a fixed fraction ($\eta = 1/64$) of it, clamped to $[1\text{ s}, 1\text{ h}]$ — Eq. (3). Coarse on a cruise, fine through a flyby, and deterministic, because the step size is a pure function of state and ephemeris.

Two details of that commit set the model’s character. First, steps land *exactly* on maneuver-node times, so a plotted burn executes in the projection at the same instant the live loop fires it — without that, plans and reality drift apart at every burn and the plotting table lies. Second, the commit message records the road not taken: universal-variable Kepler propagation and patched conics, the NASA mission-planning classic, rejected because they assume one attractor per arc and would disagree with the integrator exactly at flybys, where the game happens. The acceptance test priced the compromise: $< 5 \times 10^7$ m divergence from $\Delta t = 1$ s truth over a 20-day mid-course-burn transfer, against a capture threshold of 10^9 m.

3.3 The model learns to carry passengers (M7, M14)

The Electric Universe layer added plasma forces and hull charge, and immediately collided with the adaptive step: a naive charge-relaxation update goes unstable when Δt is an hour. The fix — a clamped exponential approach to ambient charge, $\Delta q = (q_{\text{amb}} - q) \min(1, \Delta t/\tau)$ — was the first time a feature had to be written *for* the integrator rather than merely on top of it, and it set the rule every later force followed: stable at any step the planner might take.

The owner’s playtests then calibrated what the model had to be good *for*. His standard became a Core test with his metric in its name, `SaturnSail_FitsInOneNavDeskSitDown`: one plotted plan, entered at the nav desk in one sitting, must reach Saturn’s port zone inside a 730-day horizon — and a Mercury hard-brake transfer must fit the same sit-down. The integrator’s accuracy budget is not an abstract tolerance; it is “the perihelion of the plotted ellipse kisses Mercury’s orbit ring on the owner’s screen.”

3.4 The owner’s question that merged two integrators (M19)

For eighteen milestones the game ran two integrators: the adaptive one for planning and the fixed one-second loop for living. Then the owner, mid-voyage at $10000\times$ warp, asked in one sentence what should have been obvious: “*since it already knows the trajectories from planning, why is it so heavy running?*” He was exactly right. At $10000\times$ the live loop performed 10,000 gravity-plus-plasma evaluations per real second for the player ship alone, nearly all of them in deep space where the dynamical time is measured in weeks.

The fix was to let the live simulation *become* its own projection: `RunAdaptive` — the planner’s stepping, Eq. (3), exact node landing included, returning only the final state. Above $100\times$ warp the time accumulator is consumed in fixed 60-second quanta through it; fixed quanta keep the trajectory independent of frame timing, preserving determinism. Below $100\times$, nothing changed, byte for byte. Measured headless with plasma on and full traffic: 9,969 simulated seconds per real second at $10000\times$ — effectively the full requested rate, in a browser, in interpreted IL. The projection method and the game loop have been the same code path since; matching the fixed integration within 0.1% over a 30-day cruise with a mid-course burn is a regression test.

3.5 The labs get the last word

The model’s most recent changes were forced, not chosen: Lab 06 proved the sampled closest-approach scan could tunnel through a periapsis and Lab 08 proved the uncertainty cone lied after a burn (§6.2); the closed-form segment minimum of Eq. (5) and the impulse term of Eq. (7) are the repairs. The pattern across all five subsections is the same one: every piece of the current model exists because something honest — a test, a lab, or the owner at $10000\times$ warp — caught the previous model being wrong or wasteful. Nothing in the integrator is there because it was elegant in the abstract; it is there because the $100\times$ interpreter tax and a two-year plotting horizon left no room for anything that was not.

4 One Simulation, Eight Desks

The ship is crewed, not driven. One deterministic simulation is projected onto eight duty stations: *Captain* (standing orders, ship status board), *Nav* (map, plotting table, orbit-assist

autopilot), *Sensors* (telescope ledger, passive watch, the eyes race), *War room* (tactical circle, intercept clock, fire control), *Trade, Comms* (departures board, dark web), *Galley* (news feed, rum), and *Deck* (a walkable interior whose raycast first-person windows show the real ephemeris sky).

Two interface rules survived playtesting. First, a desk's own topic owns roughly 70% of the screen; every other station rides along as a one-line "chip" — a current-objective summary, never a stat dump — with the captain's chip leading everywhere. Second, honest instruments are themselves game-play: sun-glare detection asymmetry makes anti-sunward the pirate's hunting angle; a charged hull glows farther than it sees; and the mutual-visibility readout ("do we see them, or do they see us first?") turns the sensor model into strategy.

5 The Game That Grew on the Core

The physics of §2 is the substrate; the last stretch of development was mostly *game* built on top of it, and almost all of it came from a single recurring playtest complaint. The owner would fly a full mission end to end — take a contract at a bar, plot a cross-well transfer, park at a moon, deliver a parcel — and report the same diagnosis every time: *the flying worked; the ship never said what it was doing*. The mechanics below are the answers, and each one obeys the project's honesty law: the game states its own state, and states it in numbers a probe measured.

5.1 A ship that speaks

The ship now narrates its own flight through one authoritative voice. A single plan-status builder feeds a NOW/NEXT banner ("NOW: coasting the transfer arc · NEXT: insert at 313 km, ~2 h") that scrolls to the active step as each becomes current, so the plan and the instruments can never disagree — there is one source of truth. Underneath it runs an edge-triggered, acknowledgeable alert channel that the banner strip, the desk chips, the ledger, and the in-fiction parrot all read from: rocks-ahead collision alarms, an amber fuel warning at the reserve line and red when no pump is reachable, and an orbit-degrading warning as the backstop. Burns — manual, plan-node, autopilot, or scheduled — announce themselves with a wall-clock-timed thruster flash and rumble that even 10,000× warp cannot hide, and mission completion earns a genuine payoff instead of coin silently appearing: a celebration, a grateful contact, and the first entry in a relationship that later mechanics build on.

5.2 Autopilot that keeps the orbit

The owner made orbit safety a design law: armed auto-orbit must end in a *kept* orbit, not merely an achieved one — "the orbit keeping should always be the job of autopilot this close to a moon, never a manual task." The autopilot therefore holds the park with periodic station-keeping trims and, decisively, *quotes their cost at arm time*: the banner over a held

orbit reads "AUTOPILOT HOLDS THE ORBIT — ENCELADUS, 313 KM, TRIM ≈ 27 P/DAY," where the 27 pulses/day is not a tuned constant but the figure Lab 25 measured for Enceladus's particular tide (§6.4). The degradation alarm is the backstop that fires only if keeping fails or the tank runs dry.

5.3 Deliberate piracy, resolved by dice

Piracy is now an act the captain *commits*, never one that happens automatically: hostile boarding and plunder require an explicit authorization, the same are-you-sure the weapons already demanded, because "hostile acts are never automatic." The consequence side is the catch encounter. When a heat-hunter's honest catch test (catch radius and relative speed, §4) flips, warp yanks to 1×, the ship is grappled, and a BUSTED pop-up opens in the collector's voice with three options — *submit* (they confiscate a heat-scaled share of what they can see), *bribe*, or *resist* (a dice-run last stand). The resolution runs on a single Core dice/modifier rule — a deliberate, cheap tabletop-style mechanic for the outcomes we chose not to deep-code, with the roll and its modifiers shown on screen. Two design laws keep it humane: it is *never* game over — a mercy rule guarantees confiscation leaves enough fuel to reach the nearest pump plus a berth fee, computed from the same fuel-reachability service the fuel alarm uses — and the RESIST/RUN odds quoted in the pop-up are Lab 27's measured pursuit numbers, not vibes.

5.4 Buried treasure, the favor bank, and a durable life

The law takes what it can *see*, which turns hiding loot into strategy. Three mechanics form one economy of consequence. *Buried treasure*: at any landable body in shuttle range, the shuttle-bay airlock gains a "bury a chest" destination that caches coin or cargo on the surface, invisible to confiscation; burying generates a full-screen *treasure map* — a captioned image of the body with a bearing and pace-count from its best landmark ("PHOBOS — FROM THE MONOLITH, 40 PACES ANTI-SPINWARD"), and, per the owner's Indiana Jones wink, a big red X that is *always* exactly right. The same code path powers a "fetch a cache" mission kind, where a contact hands you a map to someone else's hoard. *The favor bank*: a contact with real mission history can wire anonymized fuel money when you are broke at a pump (with strings — a fee, an interest-bearing debt, or a favor owed), and you can deposit your own coin with a trusted contact, invisible to confiscation like buried cargo — a goodwill you can literally bank on, tracked as one signed balance on the contact ledger. And *the vault*: the pirate's life — relationships, balances, caches, maps, insurance — persists as versioned, field-tolerant JSON plus a checksum, so an old save loads the parts it understands forever and a tampered one loads anyway but says so. It is not a physics snapshot; it is a life made durable.

5.5 The door family and the fuel gauge

Two smaller systems round out the world. A car-gauge-plain fuel economy makes reaction mass a visible trade item (“FILL HER UP”), with pulse depots that ride the rails at planets, stations, and havens but never at an ordinary moon — the gap that Lab 28 turned into an honest reachability verdict. And a family of walkable “doors” gives the ship’s interior continuity with the sky outside: the docking tube, the shuttle-bay airlock flown as a short piloted hop, and (on the board for Phobos) a beanstalk whose two ends are paired walkable ports — each door a transition the player understands as a flight, not a menu.

6 Secretly a Classroom

6.1 The Gravity Lab

The repository ships a growing ladder of type-it-in lessons — roughly thirty at this writing (labs/) — each a console probe referencing the game’s own `SpaceSails.Core`: motivation, the standard-textbook treatment (with Curtis pointers), what the game simplifies away, a numerical experiment with real printed output, and “break it yourself” exercises. The honesty rule is absolute: every printed number came from running that lesson’s probe; rerun and re-paste, never hand-edit.

6.2 The labs falsify the game — and the game gets fixed

The original prediction cone grew as

$$w(\Delta t) = w_0 + \sigma_v \Delta t + \frac{1}{2} a_{\max} \Delta t^2, \quad (6)$$

with w_0 the present track uncertainty, σ_v the velocity uncertainty, and $a_{\max}$ a bound on unmodeled acceleration. Lab 08 measured Eq. (6) against a target that actually burns and found it *flatly excluded the truth for hours after a real burn*: conservatism ratio $0.5\times$ at both 2 h and 6 h — the cone claimed half the width the evidence demanded. The fix adds an impulse term growing linearly at the plausible burst Δv ,

$$w(\Delta t) = w_0 + \sigma_v \Delta t + \frac{1}{2} a_{\max} \Delta t^2 + \Delta v_{\text{burst}} \Delta t, \quad (7)$$

and shipped as a prerequisite of fire control, with the lab’s dishonesty table reproduced verbatim as a regression test.

Lab 06 caught the older sampled closest-approach scan stepping clean through a periapsis: 380,845 km reported versus 157,866 km true — a $2.4\times$ error at exactly the moment a player cares — and priced the fix. The closed-form segment minimum of Eq. (5) is that fix, and ordnance hit resolution inherits it.

The point for this venue: the curriculum is not documentation *about* the engine; it is an adversarial test suite the engine must survive, written to be read.

6.3 Fire control as the shooting method

A firing solution is a two-point boundary value problem: leave the muzzle now, occupy the predicted target point at t_{hit} . The unknowns are launch bearing θ and ejection charge q ; the residual is the miss vector at arrival,

$$\mathbf{R}(\theta, q) = \mathbf{x}(t_{\text{hit}}; \theta, q) - \hat{\mathbf{x}}_{\text{tgt}}(t_{\text{hit}}), \quad (8)$$

where $\mathbf{x}(t_{\text{hit}}; \theta, q)$ is obtained by one full flight of the real integrator of §2 — no closed-form ballistics, no cheating. The solver is a damped 2×2 Newton iteration,

$$\begin{pmatrix} \theta \\ q \end{pmatrix}_{m+1} = \begin{pmatrix} \theta \\ q \end{pmatrix}_m - \lambda J_m^{-1} \mathbf{R}_m, \quad J_m = \left. \frac{\partial \mathbf{R}}{\partial (\theta, q)} \right|_m, \quad (9)$$

with J_m estimated by finite differences (each column one more integrator flight) and $\lambda \in (0, 1]$ a damping factor that backs off when a step would overshoot. This is the shooting method, verbatim, from any numerical-methods course — and the game plays it back honestly: the gun deck replays the iteration trace one Newton step per beat (CALCULATING FIRING SOLUTION...), locks at $T-60$ s, slews the hull, fires, and returns the helm — the Norden-bombsight beat, earned rather than faked.

Dispersion is not a tuning knob. The shot’s uncertainty *is* the target track’s cone half-width at arrival,

$$\sigma_{\text{hit}} = w_{\text{tgt}}(t_{\text{hit}}) \quad (10)$$

with w_{tgt} from Eq. (7), so fire-control quality *is* track quality: telescope work at the Sensors desk directly buys tighter shots at the gun deck — one weapon system distributed across two stations. Each locked solution prints its “gunner’s lesson” (lead angle, time of flight): the solver as flight instructor, teaching the same intuition manual intercepts demand.

6.4 The lab-to-game pipeline: every number a probe printed

Fire control (§6.3) is the sharpest instance of a general method, not a one-off. The lab course is a working R&D bench: when the game needs a new behavior, a lab *measures* it first against the shipping engine, and only then does the game ship the measured number. The pattern is worth stating plainly because it is the paper’s most reusable idea — *measure in a lab, then ship the measured number* — and the newest labs exist because a playtest exposed a mechanic the game was faking. Labs 21–30, by name and one-line finding:

- **Lab 21, “The Commuter”** (*reserved*): an Earth↔Mars Persephone cyler computed honestly in the rails, to land beside the free-return lesson.
- **Lab 22, “The air brake”**: the one new Core ingredient the flight assists needed — an exponential atmosphere with a single ballistic-coefficient knob; the Apollo skip’s capture-without-burn-up corridor is ~ 20 km of periapsis altitude wide, and a fuel-out arrival is turned bound ($114 \rightarrow 7R_J$) on drag alone.

- **Lab 23, “The moon run”**: the autopilot learns orbital mechanics — Wednesday’s reset loop burned 677 pulses fighting Saturn; planning in the giant’s frame (a 36.8 h transfer window whose porkchop floor *is* Hohmann) flies the same hop for **96 pulses**, a $7.1\times$ cut.
- **Lab 24, “The last mile”**: “almost there” is its own orbital problem — point-and-throttle chasing costs 4.0 km/s and arrives too hot to stop; 1960s change-your-period rendezvous doctrine closes a 92,640 km along-track gap for **2 pulses**, and TransferPlanner returns the whole cheaper-vs-sooner trade table.
- **Lab 25, “The tide that takes it back”**: the autopilot learns to *keep* an orbit. Swept in the live field, Luna and Titan have a stable park core but *Enceladus has none* — its Hill sphere is only 3.8 body radii, so an unkept park crashes within half a day. The keeping bill, priced in the game’s own pulses: Enceladus **27 p/day**, Luna **2**, Titan **3** — the exact numbers the armed autopilot now quotes (§5).
- **Lab 26, “The soft arrival”** (*reserved*): Titan aerocapture as an arrival mode, composing Labs 22 and 23.
- **Lab 27, “The getaway”**: the wolves’ honesty contract. Flown through the real thrust-only pursuit law, the killer identity $u^2/2a$ makes a grab earnable only inside $\sim$ the 300,000 km catch radius at modest flee speed — a clean pursuit cliff past which the closest pass screams through at 24–55 km/s and cannot grab; the three measured escapes (a Jupiter sling donating 12 km/s, a 1,567 m/s heat-bleed skim, a phasing juke that voids the firing solution) become the PursuitOdds the BUSTED pop-up reads.
- **Lab 28, “The pump crawl”**: can the ship still reach a fuel pump? Depots never ride an ordinary moon, so the reach to the nearest pump is 29 pulses from a Titan doorstep — already beyond the old flat 45-pulse reserve, the fuel-starve bug in one number. It ships FuelReachability.Assess, a well-aware Comfortable/Thin/CannotReachAPump verdict, and it also became the BUSTED encounter’s mercy law (never confiscate below reach-a-pump).
- **Lab 29, “The harbor pattern”**: what a safe arrival looks like. Flying a spread of inbound trajectories measures two harbor doors — a station clamp bubble that bills a pulse per $\sim 1\%$ of world speed you arrive hot with, and a moon door (the 949 km Hill sphere) where a too-hot fall genuinely impacts. The corridor is a constant-time-to-go glideslope, shipped as ApproachCorridor, the seam the banner’s NEXT row and the tutorial narration speak.
- **Lab 30, “The mass-driver timetable”**: the owner’s canon (Luna lobbing compute-core pods) becomes measured physics. A pod has zero maneuver budget, so its future is a closed-form Kepler conic; above the 2.376 km/s lunar-escape floor a retrograde lob threads perihelion

toward the Mercury compute yards, and the repeating cadence is a bus timetable read off the rail — half the pods always already in flight.

The through-line: five of these labs (23–25, 27, 29) were commissioned by a specific playtest failure and returned a specific constant — 96 pulses, 27 p/day, a 300,000 km cliff, a 949 km door — that the game now displays to the player as the honest cost of what they are doing. The curriculum is the game’s R&D department, and its output is the game’s numbers.

7 Experience Report: a Human PO and an AI Head Coder

7.1 Roles

The owner sets vision, plays every build, files corrections in plain language (“the sensors are off by default — why?”), and approves pull requests, often from a phone. The AI is a *head coder*, not a single programmer: it decomposes the owner’s intent into lanes, writes each lane a coordination brief, dispatches an implementation agent per lane, reviews and integrates the results, verifies live in the owner’s own browser, writes the PR narratives, and runs the merge train. Review is by playing, not by reading diffs alone.

7.2 Content per active day, honestly counted

We deliberately do not report a session speed record. The metric that matters for a game is how much *content* lands, and the honest denominator is *active coding days*, not wall-clock days: the project ran over a calendar span that included travel breaks with no merges at all. Counting merged pull requests by their squash-commit (#N) suffix on main¹ gives **165 merged PRs over 12 active days — 13.75 PRs per active day** — inside a 16-day calendar window. The distribution (Table 1) shows both the ramp and the honest gaps.

Volume is not the claim; the claim is that it was *reviewed* volume. Every PR was played on a fresh build, and the suite grew alongside it — from empty repository to the full game plan in the first days, and to roughly 600 deterministic Core tests and a ~ 30 -lesson lab course by this writing.

7.3 Orchestration: one lead, many agents, parallel worktrees

The throughput above is not one model typing faster; it is one *lead* model coordinating several implementation agents at once. Each agent works in its own isolated git worktree, so lanes never share a working tree and cannot clobber each

¹Method: squash merges leave no merge commits, so `git log -merges` finds nothing. We instead count first-parent commits on main whose subject ends in a (#N) pull-request suffix, one per merged PR, and group them by author date: `git log -date=short -pretty='%ad %s' origin/main | grep -E '^(#[0-9]+)$'`. Days with zero such commits are idle days and are excluded from the active-day count.

Date	PRs	Date	PRs
Jul 02	1	Jul 13	2
Jul 03	23	Jul 14	17
Jul 04	25	Jul 15	14
Jul 05	32	Jul 16	18
Jul 06	11	Jul 17	16
Jul 07	2	Jul 09–12: <i>travel</i>	0
Jul 08	4	Total	165

Table 1: Merged PRs per active day. 165 PRs / 12 active days = 13.75 per active day; the Jul 09–12 gap is a travel break, excluded from the denominator.

other mid-edit. The lead writes each lane a coordination brief with explicit ownership boundaries — which files a lane may touch, and the one-line-append-at-a-marked-anchor rule for the shared hotspot files — then merges integration-branch-first: results land on an integration branch and are played there before any button touches main. When two lanes conflict, the conflict is sent *back to its author agent* to resolve, because that agent holds the context the merge does not. The discipline is what lets review-by-playing scale: the lead’s attention is spent on intent and integration, not on typing every line.

Adversarial review is part of the loop, and it earns its keep by catching real bugs — including the lead’s own. Two representative saves: the lead once hand-stitched a CSS change with an unbalanced brace, which no compiler exists to catch (CSS has none) and which only a live bench flight revealed — now a brace-balance check after any hand-edited CSS is standing law, and the bench flight is mandatory. And on another defect the lead proposed a root-cause hypothesis that a review lane *refuted*, returning the true root cause instead. Being honest about these — an AI lead is wrong in specific, ordinary ways — is what makes the method credible rather than a demo.

7.4 The live-browser loop: reading the owner’s own tab

The tightest feedback path in the project is the lead driving the owner’s *actual* running browser tab during a playtest. Two examples show the range. In the first, the owner revealed the Derelict Roadster but could not click it; the lead read the live tab, root-caused a class filter in the object picker that excluded the Roadster’s category, shipped the fix to main, and the owner continued *the same mission* minutes later — triage-and-fix inside one sitting, no repro build required. In the second, a “phantom Dock button” appeared to lie about being dockable; the lead read the numbers off a *paused* frame’s screenshot and found the presentation, not the simulation, at fault — the same class of perception bug as the earlier “three target locks,” where the physics was right and the readout wrong. The screenshot-evidence loop — decide from the frame’s actual printed numbers, not from what the UI seems to say — is how those get caught.

7.5 Working agreements that made the pace safe

- **Determinism is law** — it is what makes review-by-playing sound, because what the reviewer plays is what the tests tested.
- **Core-first with tests**: every mechanic lands as a pure rule with a deterministic test before any UI touches it.
- **Worktree isolation & merge-train hygiene**: one agent per isolated worktree; one new file per parallel lane; one-line appends at marked anchors in the shared hotspot file; integration-branch-first before any merge button.
- **The bench flight is not optional**: it is the only compiler some bugs (a CSS brace, a perception mismatch) will ever meet.
- **No unverified claims**: every sentence in a PR body is verified in a live browser before it is written.

7.6 Failure gallery (honest)

The interpreted-WASM freeze class (hit twice); the destroyed-canvas class — a Blazor conditional unmounting a canvas leaves the renderer holding a dead context (found three times, then prevented by convention); the perception-bug class, where the simulation was right and the presentation lied (“three target locks”; the phantom Dock button); the lead’s own unbalanced CSS brace, caught only by a live bench flight and now guarded by a standing brace check; a lead root-cause hypothesis refuted by its own review lane; transient Pages deploy failures and the retry playbook; and a stacked-PR merge-train pitfall — deleting a merged base branch auto-closes its children — recovered live and folded into the plan document’s procedure.

7.7 What the AI could not do

Choose what the game should feel like. Every mechanic in §4–§6 traces to an owner sentence written in plain language — the ship’s voice, orbit-keeping as law, GTA-style BUSTED, buried treasure, the favor bank. The method’s throughput is the owner’s taste executed quickly, not replaced.

8 Related Work

Kerbal-genre orbital games adopt patched conics — piecewise two-body arcs with sphere-of-influence handoffs — trading multi-attractor truth for closed-form speed; we take the opposite stance and integrate everything, accepting Eq. (3)’s cost to keep flybys honest. The educational-games literature on “stealth learning” anticipates our disguise strategy; our addition is making the curriculum executable against the shipping engine. Determinism-first architectures descend from lockstep RTS netcode traditions. Prior AI-pair-programming experience reports describe a programmer and an assistant;

we report a different split — *product owner and head coder* — in which the human writes no code at all and the AI is not one coder but a lead orchestrating several implementation agents across isolated worktrees (§7.3).

9 Limitations and Future Work

The simulation is a 2D ecliptic plane with on-rails circular ephemerides. Multiplayer is archived pending the economy’s server-side move. Persistence now exists for the *life* but not the physics: the vault (§5) durably stores relationships, balances, caches, and maps, while the mid-flight world state is still a fresh start — deliberately, so a save is a pirate’s biography, not a checkpoint. Several labs remain reserved (the Persephone cyler, the Titan soft arrival), and the Ancients’ auto-plot deliberately rations automation to protect the pedagogy.

10 Summary

SpaceSails demonstrates that one uncompromising property — bit-determinism — can be spent three ways at once: as an engineering tool (replay, lockstep, testability), as a game-design tool (honest instruments whose admissions of ignorance are mechanics), and as a pedagogical tool (a lab course that is also an adversarial regression suite). The engine’s best fixes — the impulse-aware cone of Eq. (7), the closed-form no-tunneling test of Eq. (5) — were forced by its own curriculum, and its flagship mechanic is a numerical method played at the gun deck. On that same bench the game grew a voice, an orbit-keeping autopilot, a GTA-style catch encounter, and a buried-treasure economy — every displayed number first measured by a probe. Around that core, a two-person human-PO / AI-head-coder team — one lead orchestrating parallel implementation agents — sustained 165 reviewed, merged PRs across 12 active coding days (13.75 per active day) without lowering the review bar, because determinism made playing the build a sound review procedure. We believe both halves — the simulation-as-curriculum design and the orchestrated PO/head-coder process — are repeatable, and we offer the repository itself as the evidence.

Acknowledgments

SpaceSails was built as a holiday project, and it was possible at this cadence because of Anthropic’s generous usage allowance for the Claude models through Claude Code — 165 reviewed pull requests of head-coder and implementation-agent work, live-browser verification included. That allowance is, in a real sense, the project’s funding agency, and we acknowledge it as such. Anthropic neither directed nor reviewed this work. SpaceSails is a personal project of the first author and is not affiliated with Efima or Anthropic.

Reproducibility. Everything in this paper runs from the public repository: `dotnet test` for the evidence, `dotnet run -project labs/NN-... -c Release` for every printed number, and the game itself at the GitHub Pages deployment (<https://esoinila.github.io/SpaceSails-play>).